

Bài tập chương V

Câu 1:

a. 1. Khái niệm luồng (Thread):

Luồng là **đơn vị thực thi nhỏ hơn của tiến trình**. Một tiến trình có thể chứa nhiều luồng, và các luồng **chia sẻ cùng không gian bộ nhớ** cũng như tài nguyên của tiến trình mẹ. Tuy nhiên, mỗi luồng vẫn **hoạt động độc lập** với các luồng khác trong cùng tiến trình.

Luồng thường được sử dụng để **tăng hiệu năng** nhờ thực thi song song trong một tiến trình, đặc biệt trong các ứng dụng phân tán hoặc hệ thống có nhiều bộ xử lý.

2. Sự khác biệt cơ bản giữa luồng và tiến trình:

Tiêu chí	Tiến trình (Process)	Luồng (Thread)
Định nghĩa	Là một chương trình đang chạy, có không gian bộ nhớ và ngữ cảnh riêng.	Là đơn vị thực thi nhỏ hơn trong tiến trình, chia sẻ tài nguyên với tiến trình mẹ.
Bộ nhớ	Có không gian bộ nhớ riêng, vùng nhớ, bản đồ bộ nhớ, TLB riêng biệt.	Chia sẻ không gian bộ nhớ với các luồng khác trong cùng tiến trình.
Chi phí chuyển đổi	Cao: cần chuyển ngữ cảnh giữa các tiến trình, thay đổi MMU, TLB...	Thấp: chuyển đổi luồng thường không cần thay đổi hệ thống lỗi hay bộ nhớ.
Hiệu năng	Thấp hơn do chi phí quản lý lớn và thao tác ghi/đọc đĩa thường xuyên.	Cao hơn, vì giảm chi phí chuyển đổi và dễ phối hợp trong xử lý song song.
Tính trong suốt	Cao, do mỗi tiến trình được cách ly để bảo đảm an toàn và ổn định.	Thấp hơn, vì luồng ưu tiên hiệu năng hơn là độ cách ly giữa các tác vụ.

b. 1. Hiệu năng tăng nhờ xử lý song song (parallelism) trên máy tính nhiều bộ xử lý

- Trên máy tính **đa bộ vi xử lý (multi-core CPU)**, mỗi **luồng** có thể được gán cho một **lỗi xử lý riêng**, cho phép các luồng thực thi **đồng thời** thay vì lần lượt.

- Nếu ứng dụng chỉ có **một luồng đơn**, thì:
 - Khi luồng đó thực hiện một tác vụ bị **phong tỏa** (blocking) như chờ nhập liệu, chờ đọc/ghi file... thì **toàn bộ tiến trình bị đình trệ**, không làm được gì khác.
- Khi dùng **đa luồng**, hệ thống có thể:
 - Giao một luồng để **xử lý tính toán**.
 - Giao luồng khác để **giao tiếp với người dùng, sao lưu dữ liệu**, v.v.
 - Các luồng này có thể **chạy cùng lúc trên các CPU khác nhau**, do đó **giảm thời gian chờ đợi và tăng hiệu năng tổng thể**.

2. Ví dụ minh họa: Microsoft Excel và xử lý đồng thời giữa tính toán và nhập liệu

- Trong **Microsoft Excel**, mỗi ô tính có thể phụ thuộc vào các ô khác, khi người dùng nhập vào một ô, hệ thống sẽ tự động cập nhật các ô liên quan.
- Nếu chỉ dùng **một luồng đơn**:
 - Khi người dùng **đang nhập dữ liệu**, thì Excel **phải chờ** nhập xong mới bắt đầu tính toán.
 - Ngược lại, khi Excel **đang tính toán**, người dùng sẽ **không thể tiếp tục nhập liệu**.
- **Giải pháp đa luồng**:
 - **Luồng 1**: Giao tiếp với người dùng (bàn phím, giao diện...).
 - **Luồng 2**: Tính toán lại các công thức.
 - **Luồng 3**: Lưu dữ liệu tự động vào ổ đĩa (autosave).
- Nhờ cách này, Excel có thể **vừa nhận dữ liệu, vừa tính toán, vừa lưu**, đem lại **trải nghiệm mượt mà** và hiệu quả hơn.

c. 1. Trên máy khách: sử dụng đa luồng để che giấu độ trễ mạng

Mục tiêu:

Ảnh hưởng thời gian chờ đợi phản hồi từ máy chủ để ứng dụng không bị “đứng” trong lúc đợi kết quả.

Chiến lược:

- **Mỗi yêu cầu từ người dùng được xử lý trong một luồng riêng biệt.**
 - Khi người dùng gửi yêu cầu (ví dụ: tải một phần nội dung), máy khách khởi tạo một **luồng mới** để gửi yêu cầu đó.
 - Trong khi luồng đó đang chờ phản hồi từ máy chủ, **các luồng khác vẫn có thể thực hiện các tác vụ khác**: hiển thị giao diện, gửi yêu cầu khác, v.v.

2. Trên máy chủ: sử dụng đa luồng để xử lý đồng thời nhiều yêu cầu

Mục tiêu:

Đảm bảo máy chủ có thể **xử lý song song nhiều yêu cầu**, tránh việc bị nghẽn hoặc chậm trễ do các thao tác phong tỏa (blocking).

Chiến lược sử dụng luồng:

- **Một luồng tiếp nhận yêu cầu**: luôn ở trạng thái sẵn sàng lắng nghe kết nối mới từ máy khách.
- **Nhiều luồng con xử lý tác vụ**: mỗi yêu cầu được gán cho một luồng riêng để thực hiện tác vụ như đọc file, truy vấn CSDL, xử lý logic.

d. (a) Cài đặt luồng hoàn toàn ở mức người dùng (User-Level Threads - ULT)

Mô tả:

- Các luồng được quản lý bởi **thư viện luồng ở không gian người dùng**, không liên quan đến nhân (kernel).
- Hệ điều hành chỉ “thấy” tiến trình, không biết đến luồng nào bên trong.

Ưu điểm:

Yếu tố	Ưu điểm
Chi phí chuyển ngữ cảnh	Rất thấp. Chỉ là sao chép vài thanh ghi trong không gian người dùng, không cần thay đổi MMU hoặc TLB.
Hiệu năng	Cao nếu các luồng không gọi hàm hệ thống bị phong tỏa.

Dễ triển khai Thư viện luồng dễ viết, dễ quản lý, không cần can thiệp kernel.

Nhược điểm:

Yếu tố	Nhược điểm
Phong tỏa	Nếu một luồng gọi hàm hệ thống blocking (như đọc file, chờ socket), cả tiến trình bị phong tỏa → tất cả các luồng khác cũng phải dừng.
Không tận dụng CPU đa nhân	Hệ điều hành không thấy các luồng, nên chỉ cấp một lõi xử lý cho toàn tiến trình dù có nhiều luồng.
Hạn chế mở rộng	Không thích hợp cho ứng dụng yêu cầu đồng thời cao hoặc chạy trên hệ thống nhiều bộ xử lý.

(b) Kết hợp luồng người dùng và tiến trình hạng nhẹ (Lightweight Process - LWP)

Mô tả:

- Thư viện luồng người dùng quản lý các luồng logic.
- Mỗi **tiến trình hạng nhẹ (LWP)** do **kernel quản lý**, có thể thực thi một hoặc nhiều luồng người dùng.
- Từ góc nhìn hệ điều hành, LWP là “thread thật”, có thể được lập lịch và thực thi song song trên nhiều CPU.

Ưu điểm:

Yếu tố	Ưu điểm
Phong tỏa	Nếu một luồng bị blocking, chỉ LWP đó bị phong tỏa , các LWP khác vẫn chạy bình thường → không ảnh hưởng đến toàn tiến trình.
Tận dụng đa CPU	Kernel có thể lập lịch các LWP trên nhiều lõi xử lý khác nhau , tận dụng tối đa phần cứng.
Linh hoạt	Có thể kết hợp ưu điểm của cả luồng người dùng (chuyển nhanh) và LWP (xử lý blocking tốt).

Nhược điểm:

Yếu tố	Nhược điểm
--------	------------

Chi phí chuyển ngữ cảnh	Cao hơn so với ULT vì phải chuyển ngữ cảnh giữa các LWP trong không gian kernel (→ thay đổi MMU, TLB).
Phức tạp triển khai	Cần xây dựng cơ chế ánh xạ luồng người dùng vào các LWP → phức tạp hơn trong việc đồng bộ, lập lịch và xử lý lỗi.
Chi phí tài nguyên	Mỗi LWP cần một ngăn xếp, trạng thái riêng → tốn thêm bộ nhớ và tài nguyên hệ thống.

e. Trong một hệ thống phân tán phục vụ đọc nhiều – ghi ít (read-heavy, write-light) như dịch vụ tập tin, bạn sẽ chọn mô hình đa luồng, đơn luồng, hay máy trạng thái hữu hạn (event-driven FSM) cho thành phần xử lý trên máy chủ? Hãy đánh giá dựa trên các tiêu chí: hiệu năng, khả năng mở rộng, độ phức tạp phát triển và khả năng chịu lỗi.

1. Lựa chọn đề xuất: Máy trạng thái hữu hạn (event-driven FSM)

Trong hệ thống phân tán **read-heavy, write-light** như dịch vụ chia sẻ tập tin, **máy trạng thái hữu hạn** là **lựa chọn tối ưu** vì:

2. Đánh giá theo từng tiêu chí

Tiêu chí	Đơn luồng	Đa luồng	Máy trạng thái hữu hạn (FSM)
Hiệu năng	Thấp: xử lý tuần tự, bị phong tỏa khi đọc từ đĩa.	Trung bình - Cao: xử lý song song nhưng vẫn bị phong tỏa khi gọi hệ thống (I/O).	Rất cao: không bị phong tỏa, xử lý song song bằng cơ chế sự kiện.
Khả năng mở rộng	Rất kém: không thể phục vụ nhiều yêu cầu đồng thời.	Tốt nhưng có giới hạn do chi phí tạo luồng, ngưỡng CPU/thread pool.	Rất tốt: hoạt động tốt với số lượng lớn kết nối nhờ I/O không đồng bộ và xử lý theo sự kiện.
Độ phức tạp phát triển	Đơn giản nhất.	Tương đối đơn giản với thư viện hỗ trợ.	Phức tạp hơn: cần thiết kế rõ ràng FSM, xử lý logic sự kiện, lưu trạng thái.
Khả năng chịu lỗi	Kém: lỗi có thể ảnh hưởng toàn tiến trình.	Trung bình: lỗi trong 1 luồng không ảnh hưởng các luồng khác nếu xử lý đúng.	Tốt: do trạng thái và tiến trình tách biệt, có thể cô lập và xử lý lỗi tốt.

3. Giải thích lý do chọn FSM

- **Tính chất không phong tỏa:** trong hệ thống đọc nhiều, các thao tác I/O như đọc từ đĩa rất tốn thời gian. FSM sử dụng cơ chế đưa yêu cầu vào hàng đợi và xử lý song song bằng các tiến trình hạng nhẹ, tránh bị phong tỏa tiến trình chính.
- **Bộ nhớ đệm (cache):** FSM thường kết hợp với bộ nhớ đệm – giúp tăng tốc độ xử lý khi yêu cầu đọc lặp lại dữ liệu đã có.
- **Phản hồi theo sự kiện:** FSM lưu trạng thái yêu cầu và phản hồi khi dữ liệu sẵn sàng – giúp tránh xử lý tuần tự như đa luồng và đơn luồng.
- **Tối ưu cho mô hình đọc nhiều, ghi ít:** Vì việc ghi ít nên không cần xử lý đồng bộ phức tạp, trong khi đó đọc có thể được tối ưu nhờ cache và I/O bất đồng bộ.

Câu 2: (4)

a. Hãy định nghĩa vai trò chính của máy khách trong mô hình phân tán khách–chủ và nêu hai hình thức cung cấp dữ liệu cho máy khách (bản sao cục bộ vs truy vấn trực tiếp).

Trong mô hình phân tán kiểu **khách–chủ**, **vai trò chính của máy khách là cung cấp giao diện cho người sử dụng để tương tác với hệ thống**, giúp người dùng gửi yêu cầu đến máy chủ và hiển thị kết quả nhận được. Ngoài việc cung cấp giao diện thân thiện, máy khách còn cần đảm bảo **hiệu năng cao**, phản hồi nhanh để tạo cảm giác như làm việc cục bộ, không bị ảnh hưởng bởi độ trễ mạng hoặc xử lý máy chủ. Để hỗ trợ điều này, máy khách có thể thực hiện **tiền xử lý dữ liệu** trước khi gửi yêu cầu đến máy chủ.

Có hai **hình thức cung cấp dữ liệu cho máy khách**:

1. **Bản sao cục bộ (Local copy):**

Dữ liệu được sao chép từ máy chủ về lưu trên máy khách. Khi người dùng thao tác, máy khách truy xuất dữ liệu cục bộ để hiển thị, giúp phản hồi nhanh, không phụ thuộc vào độ trễ mạng hay tải máy chủ. Tuy nhiên, phương pháp này đối mặt với **vấn đề duy trì tính nhất quán dữ liệu** giữa máy chủ và máy khách. Hình thức này thường dùng trong các **ứng dụng di động hoặc ứng dụng dạng cửa sổ**, nơi máy khách có thành phần đồng bộ riêng để giao tiếp với máy chủ.

2. **Truy vấn trực tiếp (Remote query):**

Máy khách **không lưu trữ dữ liệu**, mà gửi yêu cầu truy vấn đến máy chủ mỗi khi người dùng thao tác. Giao thức chung (như HTTP) được sử dụng để lấy dữ liệu và hiển thị. Cách này giúp **đảm bảo tính nhất quán**, giảm gánh nặng đồng bộ, và phù hợp với mô hình **ứng dụng web** trên trình duyệt, nơi máy khách chủ yếu chỉ làm nhiệm vụ hiển thị. Đây là hình thức ngày càng phổ biến trong các hệ thống phân tán hiện đại, đặc biệt trên Internet.

b. Trong mô hình sử dụng bản sao cục bộ trên máy khách, dữ liệu của dịch vụ được sao chép từ máy chủ về lưu trên máy khách để tăng hiệu năng và đảm bảo thời gian phản hồi gần như tức thì. Tuy nhiên, cơ chế này đòi hỏi phải có một quá trình đồng bộ dữ liệu giữa máy khách và máy chủ nhằm đảm bảo dữ liệu trên hai bên luôn cập nhật và nhất quán.

Cơ chế đồng bộ dữ liệu:

- **Mỗi dịch vụ trên máy chủ sẽ có một thành phần tương ứng trên máy khách**, cho phép hai bên liên lạc và đồng bộ dữ liệu qua mạng.
- **Tầng ứng dụng trên máy khách sử dụng giao thức riêng** để kết nối và đồng bộ với máy chủ.
- Đồng bộ thường được thực hiện vào **thời điểm rỗi** của máy khách, tức là hệ thống sẽ **tận dụng các khoảng thời gian nhàn rỗi để đồng bộ dữ liệu một cách nền (background)**, giúp người dùng không nhận ra đang có quá trình trao đổi dữ liệu.

Những thách thức về tính nhất quán:

- **Dữ liệu trên máy khách có thể bị lỗi thời** so với dữ liệu gốc trên máy chủ nếu quá trình đồng bộ bị gián đoạn hoặc chậm trễ.
- Khó khăn trong việc **phát hiện và xử lý xung đột dữ liệu**, đặc biệt khi cùng một dữ liệu được chỉnh sửa cả ở phía máy khách và máy chủ cùng lúc.
- **Tính nhất quán tạm thời (eventual consistency)** là thách thức phổ biến, khi dữ liệu chỉ nhất quán sau một khoảng thời gian nhất định.

Cách máy khách "che giấu" quá trình đồng bộ:

- Thiết kế hệ thống sao cho **quá trình đồng bộ diễn ra hoàn toàn trong suốt với người dùng**, tức là **người dùng không nhận ra hoặc bị gián đoạn** bởi việc cập nhật dữ liệu.
- **Tận dụng thời gian rỗi** để thực hiện việc đồng bộ nền.
- **Sử dụng cơ chế đệm (cache)** để lưu dữ liệu mới nhất đã truy xuất, đảm bảo rằng trong trường hợp mất kết nối tạm thời, máy khách vẫn có thể cung cấp dữ liệu cho người dùng mà không bị gián đoạn.
- Khi cần thiết, máy khách có thể **lặng lẽ gửi lại yêu cầu** nếu xảy ra lỗi truyền thông hoặc **chuyển hướng đến máy chủ khác** trong hệ thống có nhân bản máy chủ, đảm bảo tính liên tục của dịch vụ.

c. 1. Khi nào đọc dữ liệu từ cache, khi nào gửi yêu cầu mới?

Trong mô hình **offline-first**, chiến lược thường dùng là:

- **Đọc từ cache trước (cache-first):**
 - Khi người dùng mở app, Service Worker sẽ **ưu tiên truy xuất nội dung HTML/JSON từ cache**, đảm bảo tốc độ phản hồi tức thì và hoạt động tốt ngay cả khi mất kết nối mạng.
 - Điều này giúp app khởi chạy nhanh và phản hồi mượt mà.
- **Gửi yêu cầu mới trong nền (network update):**
 - Sau khi dữ liệu từ cache được hiển thị, **Service Worker sẽ gửi yêu cầu HTTP đến server trong nền** để lấy phiên bản mới nhất của dữ liệu.
 - Khi dữ liệu mới nhận về, nó sẽ:
 - **Cập nhật cache** (để dùng cho lần sau).
 - **Cập nhật UI** nếu dữ liệu thay đổi rõ rệt (ví dụ bằng sự kiện hoặc thông báo nhẹ trong nền).

→ Chiến lược này còn gọi là **Stale-While-Revalidate**:

“Hiển thị dữ liệu cũ (stale) từ cache ngay lập tức, nhưng âm thầm tải dữ liệu mới (revalidate) từ server để cập nhật.”

2. Làm thế nào để tận dụng “thời gian rảnh” để đồng bộ mà không gây gián đoạn UX?

Để **đồng bộ ngầm (background sync)** trong thời gian “rảnh” mà không ảnh hưởng đến trải nghiệm người dùng:

- **Sử dụng Background Sync API** (có hỗ trợ từ Service Worker):
 - Khi người dùng thực hiện thay đổi dữ liệu **offline**, các thay đổi được **lưu tạm vào IndexedDB hoặc cache**.
 - Khi kết nối mạng sẵn sàng và app ở trạng thái “rảnh” (tức không có thao tác tương tác), Service Worker sẽ:
 - **Tự động đồng bộ dữ liệu lên server.**
 - **Gửi lại những request trước đó bị lỗi** (ví dụ: POST tạo bài viết, PATCH cập nhật...).
- Tận dụng **requestIdleCallback()** hoặc **setTimeout()** nhẹ:

- Các thao tác không khẩn cấp (như cập nhật cache hoặc tải trước tài nguyên) được thực hiện **khi trình duyệt nhàn rỗi**, tránh can thiệp vào trải nghiệm chính của người dùng.
- **Sử dụng hiệu ứng loading không gây khó chịu:**
 - Thay vì làm chậm UI để chờ đồng bộ, có thể hiển thị thông báo nhẹ (ví dụ: “Cập nhật đang được tải...”) khi thực sự cần cập nhật UI từ dữ liệu mới.

d. ♦ **Chiến lược (a): Gửi lại yêu cầu nhiều lần đến cùng một máy chủ khi có lỗi truyền thông**

Ưu điểm:

- **Tài nguyên mạng tiết kiệm hơn:** chỉ truyền qua lại với một máy chủ tại một thời điểm.
- **Đơn giản trong cài đặt và logic xử lý lỗi:** chỉ cần thiết kế retry logic đơn giản trên máy khách hoặc phần mềm trung gian.

Nhược điểm:

- **Độ trễ cao hơn khi có lỗi:** nếu máy chủ đầu tiên không phản hồi hoặc bị lỗi, máy khách phải chờ timeout rồi mới thử lại ⇒ tổng thời gian tăng.
- **Độ tin cậy thấp hơn khi máy chủ hỏng hoàn toàn:** nếu máy chủ đó down, phải thử lại nhiều lần hoặc chuyển sang máy khác, gây gián đoạn UX.
- **Không tận dụng tính nhân bản mạnh mẽ của hệ thống:** không khai thác được ưu thế từ nhiều bản sao đang hoạt động.

♦ **Chiến lược (b): Gửi song song đến nhiều bản sao và dùng kết quả trả về đầu tiên**

Ưu điểm:

- **Độ trễ thấp:** thời gian phản hồi gần như tối ưu (lấy kết quả từ máy chủ nhanh nhất).
- **Độ tin cậy cao:** nếu một hoặc vài máy chủ gặp sự cố, vẫn còn máy khác phản hồi ⇒ đảm bảo hệ thống vẫn phục vụ được.
- **Cải thiện UX trong các mạng có độ trễ không ổn định hoặc thường xuyên lỗi.**

Nhược điểm:

- **Tốn tài nguyên mạng hơn nhiều:** gửi nhiều bản sao yêu cầu, làm tăng băng thông và tải lên các máy chủ.
- **Tài nguyên máy chủ bị lãng phí:** các máy chủ xử lý nhưng kết quả không được dùng.
- **Phức tạp hơn trong xử lý dữ liệu về:** cần cơ chế hủy các request chậm hơn hoặc loại bỏ các kết quả dư thừa.

e. ♦ (i) Edge caching trên máy khách

(Lưu bản sao dữ liệu cục bộ, ví dụ: IndexedDB, Cache API qua Service Worker)

Ưu điểm:

- **Hiệu năng vượt trội:** gần như **thời gian phản hồi bằng 0**, không phụ thuộc độ trễ mạng.
- **Giảm tải máy chủ web:** mỗi máy khách có thể phục vụ chính mình mà không cần gửi yêu cầu liên tục.
- **Offline-first UX:** vẫn dùng được khi mất mạng, rất hữu ích cho mobile app, SPA, progressive web app (PWA).
- **Tối ưu tài nguyên mạng:** giảm lượng dữ liệu truyền đi.

Nhược điểm:

- **Tính nhất quán khó kiểm soát:** dữ liệu cache có thể lỗi thời nếu không đồng bộ kịp thời.
- **Chi phí lưu trữ phía máy khách:** cần bộ nhớ cục bộ, phải xử lý thiếu dung lượng hoặc giới hạn trình duyệt.
- **Bảo mật dữ liệu nhạy cảm:** khó kiểm soát nếu người dùng jailbreak, root hoặc truy cập trái phép vào máy.

♦ (ii) Cache chỉ trên máy chủ web, fetch mới từ máy khách

Ưu điểm:

- **Tính nhất quán cao hơn:** luôn truy cập dữ liệu gần nhất từ máy chủ (hoặc cache gần backend).
- **Kiểm soát bảo mật dễ hơn:** dữ liệu chỉ lưu ở server-side, không lo lộ cache trên client.
- **Triển khai đơn giản hơn:** máy khách ít logic, chỉ hiển thị.

Nhược điểm:

- **Hiệu năng phụ thuộc mạng:** mỗi lần truy cập đều phải fetch $\Rightarrow$ tăng độ trễ.
- **Gây tải lớn lên server khi có hàng triệu máy khách.**
- **Không hỗ trợ offline:** trải nghiệm bị gián đoạn nếu mạng chập chờn.

Khuyến nghị:

- **Với các hệ thống yêu cầu hiệu năng cao, offline support, hoặc giảm tải server** (như: ứng dụng mobile, mạng xã hội, nội dung tĩnh):
 $\rightarrow$ **Nên sử dụng edge caching**, nhưng kết hợp với cơ chế đồng bộ ngầm, xác thực token và mã hóa local cache (ví dụ: AES + IndexedDB + Service Worker).
- **Với các hệ thống yêu cầu tính nhất quán và bảo mật dữ liệu là ưu tiên hàng đầu** (như: ngân hàng, quản lý y tế, thông tin nhạy cảm):
 $\rightarrow$ **Nên dùng chiến lược (ii)** — máy chủ kiểm soát, client là “thin client”.
- **Giải pháp lai (hybrid) là tối ưu:**
 - Dữ liệu tĩnh, ít thay đổi (UI template, metadata, ảnh) $\rightarrow$ cache trên máy khách.
 - Dữ liệu động, cá nhân hóa, cần bảo mật $\rightarrow$ fetch từ server mỗi lần.

Câu 3: (5)

a. • Định nghĩa khái niệm di trú mã (code migration) trong hệ thống phân tán và phân biệt nó với gọi thủ tục từ xa (RPC):

Di trú mã là quá trình di chuyển một chương trình hoặc một đoạn mã (có thể là mã nguồn hoặc mã nhị phân) từ một máy tính này sang một máy tính khác trong hệ thống phân tán, nhằm tăng hiệu năng và tính linh hoạt cho hệ thống. Sau khi được chuyển đến máy đích,

đoạn mã có thể tiếp tục thực thi một cách **độc lập, không phụ thuộc vào kết nối mạng** với máy gốc.

Ví dụ điển hình là khi một lượng dữ liệu lớn nằm trên máy chủ, thay vì truyền toàn bộ dữ liệu đó về máy khách để xử lý (tốn băng thông), ta có thể gửi đoạn mã xử lý từ máy khách lên máy chủ để thực thi tại chỗ và chỉ truyền kết quả về. Hoặc trong các hệ thống tìm kiếm phân tán, có thể dùng một "agent" di trú qua nhiều trang để xử lý cục bộ tại từng nơi.

Ngược lại, **gọi thủ tục từ xa (RPC - Remote Procedure Call)** là kỹ thuật cho phép một tiến trình trên một máy tính gọi thực thi một hàm hoặc thủ tục nằm trên một máy tính khác như thể nó đang gọi nội bộ. Tuy nhiên, RPC **phụ thuộc vào kết nối mạng liên tục**, và nếu mất kết nối trong khi đang thực hiện thủ tục thì quá trình sẽ bị lỗi hoặc gián đoạn.

So sánh di trú mã và RPC:

Tiêu chí	Di trú mã (Code Migration)	Gọi thủ tục từ xa (RPC)
Mục đích chính	Tăng hiệu năng, giảm dữ liệu truyền qua mạng	Gọi hàm/tác vụ từ xa theo cách gọi nội bộ
Độc lập kết nối mạng	Có (mã tiếp tục chạy sau khi di chuyển)	Không (phụ thuộc kết nối mạng liên tục)
Dạng hoạt động	Chuyển mã đến máy khác để chạy	Gọi hàm từ xa, thực thi trên máy khác rồi trả kết quả
Chi phí hệ thống	Cao hơn (do di chuyển tiến trình, đồng bộ trạng thái...)	Thấp hơn (chỉ là gọi hàm từ xa)
Tình huống sử dụng	Khi xử lý tại chỗ gần dữ liệu, xử lý song song, giảm tải mạng	Khi cần tương tác liên tục giữa client/server

b. Trình bày khác biệt giữa di trú yếu và di trú mạnh (weak vs. strong migration). Cho ví dụ thực tế cho mỗi loại và nêu tại sao di trú mạnh phức tạp hơn.

1. Khái niệm và phân biệt:

Đặc điểm	Di trú yếu (Weak Migration)	Di trú mạnh (Strong Migration)
Thành phần được di trú	Chỉ di chuyển phần mã và dữ liệu khởi động	Di chuyển cả phần mã, trạng thái thực thi, và ngữ cảnh tiến trình
Vị trí bắt đầu thực thi	Luôn khởi động lại từ đầu hoặc từ điểm xác định trước	Tiếp tục từ nơi tiến trình đã tạm ngừng
Đơn giản hay phức tạp	Đơn giản, dễ triển khai	Phức tạp do phải lưu và khôi phục trạng thái tiến trình

Cách thực hiện phổ biến	Tải đoạn mã mới về máy đích và khởi chạy lại tiến trình	Tạm dừng tiến trình → ghi lại trạng thái → di chuyển → khôi phục và tiếp tục chạy
Ứng dụng phổ biến	Nâng cấp phần mềm, kiểm tra dữ liệu ở phía client	Tác tử di động, các hệ thống tính toán linh hoạt phân tán

2. Ví dụ thực tế:

- **Ví dụ về di trú yếu:**

Trong các ứng dụng web, khi người dùng mở một biểu mẫu nhập liệu, trình duyệt có thể tải về một đoạn mã JavaScript từ máy chủ để **kiểm tra dữ liệu đầu vào** (như định dạng email, số điện thoại, v.v). Đoạn mã đó được thực thi ở phía **máy khách**, giúp **giảm tải cho máy chủ**. Đây là di trú yếu vì chỉ phần mã được tải xuống và thực thi từ đầu.

- **Ví dụ về di trú mạnh:**

Một **tác tử tìm kiếm (mobile agent)** di chuyển giữa nhiều máy chủ web để thu thập thông tin. Khi nó rời một máy chủ, nó **ghi lại trạng thái hiện tại** (ví dụ: đang truy xuất đến trang thứ mấy), rồi di chuyển sang máy kế tiếp và **tiếp tục thực hiện tại điểm dừng**. Đây là di trú mạnh vì cần bảo toàn và khôi phục **toàn bộ ngữ cảnh thực thi**.

3. Tại sao di trú mạnh phức tạp hơn?

- Cần **lưu lại và truyền tải trạng thái thực thi** (gồm ngăn xếp, biến cục bộ, bộ đếm chương trình...).
- Phải đảm bảo **khôi phục đúng ngữ cảnh** tại máy đích sao cho tiến trình tiếp tục mà không gây lỗi logic.
- Cần tương thích hệ điều hành, phần cứng, hoặc máy ảo giữa các máy.
- Phải giải quyết **vấn đề liên kết với tài nguyên ngoài** (file, socket, cổng mạng...) mà không gây gián đoạn.

c. Chiến lược di trú mã

1. Di trú từ máy khách lên máy chủ:

- **Phần nên di trú:**

Các đoạn mã xử lý dữ liệu (tính toán, lọc, phân tích, thống kê...) nên được **chuyển từ máy khách lên máy chủ**.

- **Loại di trú:**
Di trú yếu
- **Giải thích:**
 - Máy chủ **đã chứa toàn bộ dữ liệu**, nên việc xử lý tại máy chủ là tối ưu.
 - Di trú yếu đủ để tải đoạn mã xử lý từ máy khách lên máy chủ, sau đó thực thi từ đầu trên dữ liệu tại máy chủ.
 - Không cần lưu trạng thái hoặc tiếp tục từ tiến trình đang chạy.
 - Điều này giúp máy khách chỉ cần nhận **kết quả đầu ra**, giảm thiểu băng thông mạng.
- **Ví dụ thực tế:**
Đoạn mã xử lý biểu đồ, thống kê hoặc tính trung bình/độ lệch chuẩn của một tập dữ liệu lớn — được gửi từ client lên server dưới dạng script (ví dụ Python/R).

2. Di trú từ máy chủ xuống máy khách (nếu có):

- **Phản nên di trú:**
Các đoạn mã **kiểm tra dữ liệu đầu vào** hoặc **xử lý UI nhẹ**, ví dụ: xác thực định dạng, kiểm tra logic đơn giản trước khi gửi lên server.
- **Loại di trú:**
Di trú yếu
- **Giải thích:**
 - Máy chủ có thể đẩy các đoạn mã kiểm tra sang client để giảm lượt gửi yêu cầu không hợp lệ.
 - Việc này giúp giảm tải cho máy chủ và tiết kiệm băng thông mạng không cần thiết do gửi dữ liệu sai định dạng.
 - Không cần khôi phục trạng thái, chỉ thực thi lại từ đầu khi tải xuống.
- **Ví dụ thực tế:**
JavaScript kiểm tra định dạng email, số điện thoại, bắt buộc điền ô dữ liệu...

3. Có cần di trú mạnh không?

- Trong bài toán **giảm băng thông mạng**, **di trú mạnh không cần thiết**, vì:
 - Không cần khôi phục trạng thái thực thi giữa hai máy.

- Việc tạm dừng và tiếp tục tiến trình từ nơi dừng lại **chỉ nên dùng trong các tác tử di động hoặc hệ thống song song phức tạp**.
- Di trú mạnh **phức tạp hơn nhiều**, đòi hỏi khả năng lưu/ngắt/khôi phục trạng thái, không cần thiết cho mục tiêu chính là tiết kiệm băng thông.

d. Dựa trên ba mức *mức độ tham chiếu tài nguyên* (mạnh, yếu, rất yếu) và ba *mức gắn kết tài nguyên* (cố định, cao, thấp), phân tích thách thức khi di trú mã có tham chiếu đến:

- 1. Một tập cấu hình trên đĩa (gắn kết thấp).**
- 2. Một socket kết nối TCP hiện tại (gắn kết cố định).**

1. Trường hợp: Một tập cấu hình trên đĩa (gắn kết thấp)

Đặc điểm:

- **Tài nguyên:** Tập cấu hình trên đĩa.
- **Gắn kết:** **Thấp** – dễ dàng di chuyển hoặc sao chép.
- **Tham chiếu:** Có thể là **mạnh** (nếu đường dẫn tuyệt đối), **yếu** (nếu chỉ cần nội dung), hoặc **rất yếu** (nếu chỉ cần loại định dạng tập).

Thách thức và giải pháp theo mức tham chiếu:

Mức tham chiếu	Thách thức chính	Giải pháp phù hợp
Mạnh	Đường dẫn tập có thể không tồn tại hoặc không giống nhau trên máy đích.	Thiết lập tham chiếu toàn cục (GR) hoặc di chuyển file (MV) .
Yếu	Chỉ cần nội dung tập, không cần đúng đường dẫn .	Sao chép nội dung tập (CP) là đủ.
Rất yếu	Chỉ cần có một tập cùng loại (ví dụ: .conf) để ghi.	Tiến trình nhúng lại tài nguyên cục bộ (RB) .

2. Trường hợp: Một socket kết nối TCP hiện tại (gắn kết cố định)

Đặc điểm:

- **Tài nguyên:** Kết nối TCP đang hoạt động (socket).
- **Gắn kết:** **Cố định** – bị ràng buộc địa chỉ IP, cổng mạng, không thể di trú theo cách thông thường.

- **Tham chiếu:** Thường là **mạnh** (nếu giữ nguyên kết nối), hoặc **yếu** (nếu chỉ cần trao đổi dữ liệu).

Thách thức và giải pháp theo mức tham chiếu:

Mức tham chiếu	Thách thức chính	Giải pháp phù hợp
Mạnh	Kết nối ràng buộc địa chỉ IP, cổng → không thể mang đi được .	Thiết lập tham chiếu toàn cục (GR) và dùng máy cũ làm proxy (→ gây trễ, rủi ro).
Yếu	Nếu chỉ cần dữ liệu đang truyền, có thể tạo lại kết nối mới nhưng sẽ mất trạng thái phiên .	Khó giải quyết trực tiếp, cần thiết kế lại ứng dụng sao cho không phụ thuộc phiên.
Rất yếu	Gần như không áp dụng được – socket là tài nguyên phiên liên tục.	Không khả thi – cần tránh ràng buộc di trú mã với socket ở mức này.